3. Voice Control Muto Line Following Autonomous Driving

Before running this program, you need to bind the port numbers of the voice board and the ROS expansion board on the host machine. You can refer to the previous chapter for binding. When entering the Docker container, you need to mount this voice board so that it can be recognized in the Docker container.

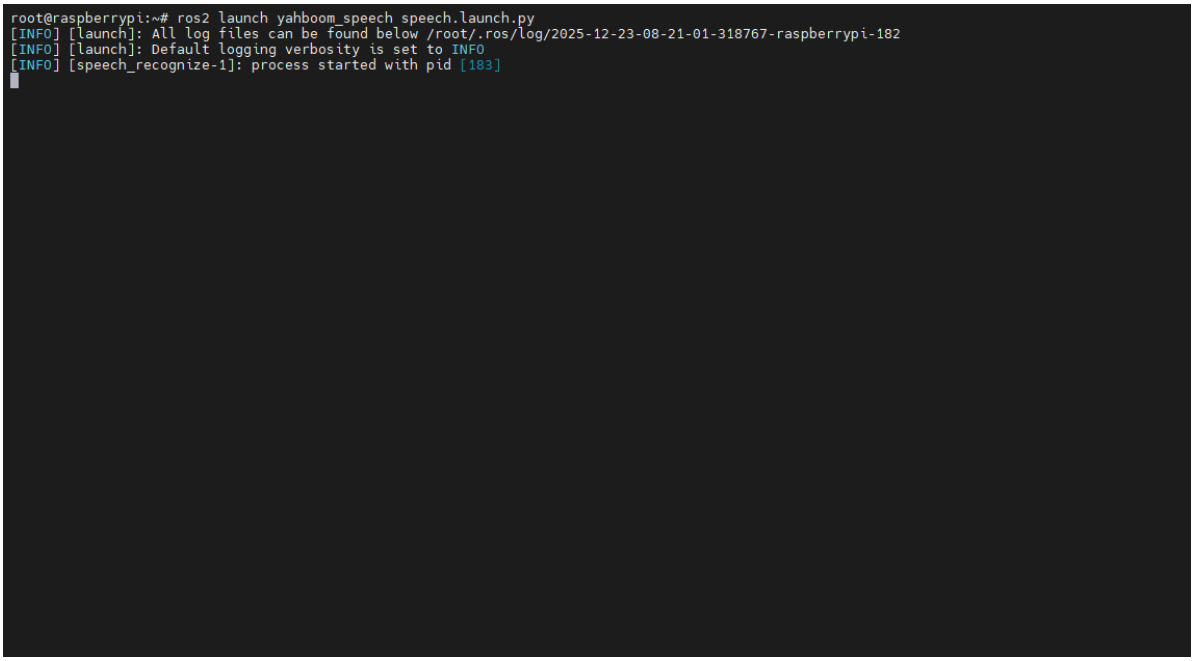
1. Program Function Description

After the program starts, say "Hello, yahboom" to the module. The module will reply with "I'm here" to wake up the voice board. Then you can tell it to follow a line of a certain color, such as "track red line," "track green line," "track blue line," or "track yellow line." After receiving the command, the program recognizes the color and loads the processed image. Muto will start moving along the route of the recognized color. During the autonomous line-following process, if it encounters an obstacle, the buzzer will beep "didi di" and the car will stop.

2. Program Code Reference Path

After entering the Docker container, start the voice control node:

```
ros2 launch yahboom_speech speech.launch.py
```



3. Program Startup

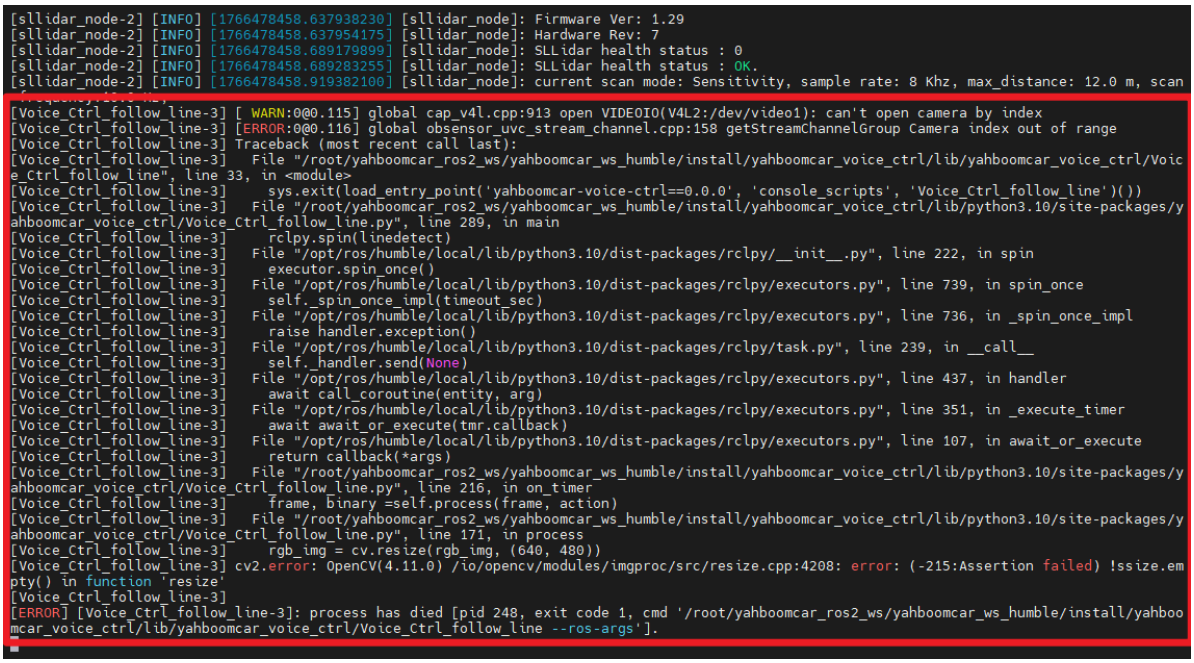
3.1. Startup Command

Create a new terminal, enter the Docker container, and then enter in the terminal:

```
ros2 launch yahboomcar_voice_ctrl voice_ctrl_follow_line_launch.py
```

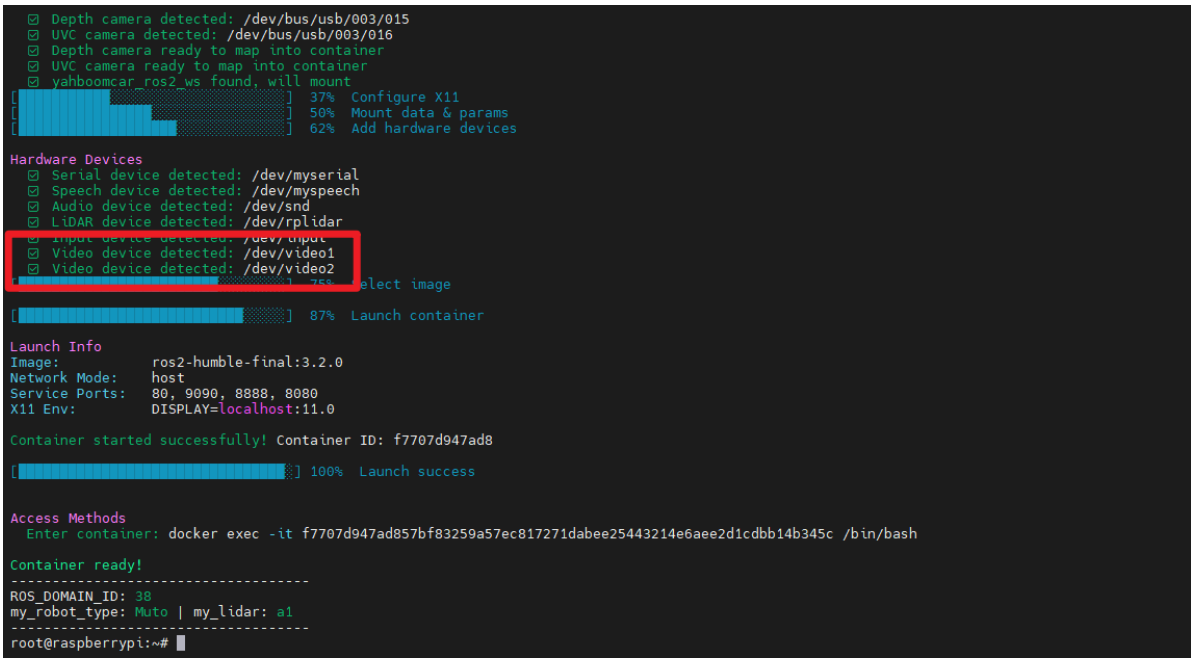
Error Handling

If the following error occurs, it means the system has not recognized the camera device. You need to check if video1 and video2 were successfully mounted when Docker started.



We need to unplug and plug in the USB camera again, and restart the Docker image.

Terminal 1:




```
ros2 launch yahboom_speech speech.launch.py
```

Terminal 2:

```
enter_muto_docker
```

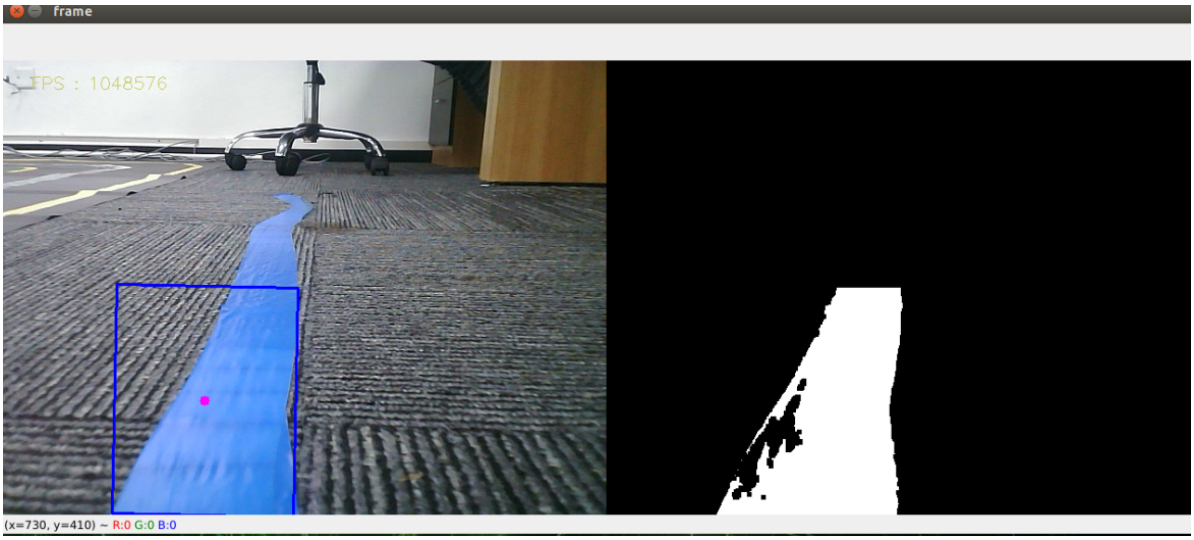
```
ros2 launch yahboomcar_voice_ctrl voice_ctrl_follow_line_launch.py
```

```
pi@raspberrypi:~ $ ./enter_docker.sh
正在进入容器 f7707d947ad8 的终端...
-----------------------------------
ROS_DOMAIN_ID: 38
my_robot_type: Muto | my_lidar: a1
-----------------------------------
root@raspberrypi:/# ros2 launch yahboomcar_voice_ctrl voice_ctrl_follow_line_launch.py
[INFO] [launch]: All log files can be found below /root/.ros/log/2025-12-23-08-31-21-846321-raspberrypi-191
[INFO] [launch]: Default logging verbosity is set to INFO

-------- rplidar_type = a1 --------

[INFO] [muto_driver-1]: process started with pid [192]
[INFO] [sllidar_node-2]: process started with pid [194]
[INFO] [Voice_Ctrl_follow_line-3]: process started with pid [196]
[INFO] [joy_node-4]: process started with pid [199]
[INFO] [yahboom_joy-5]: process started with pid [201]
[sllidar_node-2] [INFO] [1766478682.059420295] [sllidar_node]: SLLidar running on ROS2 package SLLidar.ROS2 SDK Version:1.0.1, SLLIDAR SDK Ve
rsion:2.0.0
[sllidar_node-2] [INFO] [1766478682.118716761] [sllidar_node]: SLLidar S/N: 6A97EDF9C7E29BD1A7E39EF2FA44431B
[sllidar_node-2] [INFO] [1766478682.118835357] [sllidar_node]: Firmware Ver: 1.29
[sllidar_node-2] [INFO] [1766478682.118859172] [sllidar_node]: Hardware Rev: 7
[sllidar_node-2] [INFO] [1766478682.170333383] [sllidar_node]: SLLidar health status : 0
[sllidar_node-2] [INFO] [1766478682.170457312] [sllidar_node]: SLLidar health status : OK.
[sllidar_node-2] [INFO] [1766478682.402916746] [sllidar_node]: current scan mode: Sensitivity, sample rate: 8 Khz, max_distance: 12.0 m, scan
 frequency:10.0 Hz,
```

Point Muto's camera downwards so it can see the line. First, wake up the module ("Hello, yahboom"). After receiving a reply, to track the blue line, you can say "track blue line".



The car will immediately start following the line. At this time, press the R2 button on the controller or the spacebar on the keyboard to stop line following.

3.2. Adjusting HSV Values

The HSV values loaded in the program are not suitable for all scenarios. As is well known, lighting has a significant impact on the results of image processing. Therefore, if the image processing effect with the loaded HSV values is not good, you need to recalibrate the HSV values. The calibration method is as follows:

- Run the line-following program and the dynamic parameter reconfigure tool:

  ```
  ros2 run rqt_reconfigure rqt_reconfigure
  ```

- Press the 'r' key on the keyboard to enter color selection mode. In the area where you want to follow the line, draw a box to select a region. Then, click on the blank area of the reconfigure_GUI interface. You will notice that the HSV values inside change. Update these values in `self.hsv_range` in `Voice_Ctrl_follow_line_a1.py`, making sure not to mix up

the colors.

- Finally, after modifying the `Voice_Ctrl_follow_line_a1.py` code, you need to return to the `yahboomcar_ws` directory, compile with **colcon build**, and source with **source install/setup.bash**.

4. Core Code

The principle here is the same as in the previous "XI. OpenCV Series Course - 6. Line Following Autonomous Driving". The speed is calculated based on the center coordinates of the processed image. The only addition is loading the HSV values via voice command. Previously, you had to manually select the line color with the mouse, but with the voice module, we only need to load the corresponding HSV values based on the command. The core content is as follows:

```
def process(self, rgb_img, action):

    binary = []
    rgb_img = cv.resize(rgb_img, (640, 480))

    if self.img_flip == True: rgb_img = cv.flip(rgb_img, 1)
    # Start receiving voice commands, publishing instructions, and loading HSV
values here
    self.command_result = self.spe.speech_read()
    self.spe.void_write(self.command_result)
    if self.command_result == 23:
    self.model = "color_follow_line"
    print("red follow line")
    # Red HSV
    self.hsv_range = [(0, 84, 131), (180, 253, 255)]
    .....
# The following part passes the HSV values, processes the image, gets a
self.circle value, and finally passes it to the self.execute function to
calculate the speed
if  self.model == "color_follow_line":
    rgb_img, binary, self.circle = self.color.line_follow(rgb_img,
self.hsv_range)
    if len(self.circle) != 0:
        threading.Thread(target=self.execute, args=(self.circle[0],
self.circle[2])).start()
```

The command words for this lesson correspond as follows:

Command Word	Voice Recognition Module Result
CLOSE-TRACKING-MODE	22
TRACK-RED-LINE	23
TRACK-GREEN-LINE	24
TRACK-BLUE-LINE	25
TRACK-YELLOW-LINE	26